TASK 8 – Plan the Algorithm Before Coding

Algorithmic problem-solving steps

1. Understand: identify inputs, outputs, rate rules, limits and payment period. 2. Plan: select numeric types, define validation, derive the rate formula and choose how totals are accumulated. 3. Write: implement small functions and a loop following the pseudocode. 4. Test: use normal, boundary and invalid cases; compare expected and actual totals.

SIMATS-style pseudocode

```
BEGIN
    SET TEN_DAY_TOTAL <- 0.00
    REPEAT for each delivery entry
        READ farmer_id, litres, fat, snf, shift
        IF litres <= 0 OR fat < 2.0 OR fat > 7.0 THEN
            DISPLAY "Invalid entry"
            CONTINUE to next entry
        ENDIF
        fat_steps <- ROUND((fat - 3.5) / 0.1)
        snf_steps <- ROUND((snf - 8.5) / 0.1)
        rate <- 30.00 + (fat_steps * 0.40) + (snf_steps * 0.30)
        payment <- ROUND(litres * rate, 2)
        TEN_DAY_TOTAL <- TEN_DAY_TOTAL + payment
        DISPLAY farmer_id, rate, payment
    ENDREPEAT
    DISPLAY TEN_DAY_TOTAL
END
```

Flowchart description

Start → Read farmer and delivery data → Are numeric values valid? If No, display an error and return to input. If Yes, check litres > 0 and fat within 2.0–7.0. If invalid, display an error and return to input. Otherwise calculate fat/SNF steps → calculate rate → calculate rounded payment → add payment to ten-day total → more entries? If Yes, read next entry; if No, display total → Stop.

Hand trace for three entries

For the examples below, calculations use the assignment's original formula and each payment is rounded to two decimals.

Trace table

Entry	Calculation	Payment	Running total
1	10 L × Rs 30.00	Rs 300.00	Rs 300.00
2	8.5 L × Rs 33.40	Rs 283.90	Rs 583.90
3	12 L × Rs 27.80	Rs 333.60	Rs 917.50

Alternative and complexity

Storing every entry in a list and summing at the end takes O(n) time and O(n) space. It is useful when records must be audited, displayed, corrected or reprocessed. Keeping a running total as each payment is calculated also takes O(n) time but only O(1) extra accumulator space. A real co-operative should still persist individual transaction records in a database for audit and dispute resolution; the running total can be maintained alongside those records.